ServoraBd

Local Service Marketplace

RENDER DEPLOYMENT RECORD

How this project was put on the internet: the code changes it needed, the two routes that failed, the one that worked, the move to Neon when Render's free database expired, and what is still outstanding.

Version	1.1
Status	Live on the free tier
Date	2 November 2026
Author	Jahid H. R.
Stack	Django · DRF · React (JS) · PostgreSQL

Contents

1 What was deployed

2 What had to change in the code

3 Two routes that did not work

4 The route that worked: services by hand

5 How it was verified

6 What is still outstanding

1. What was deployed

Piece	Address	Plan
Static site	`servorabd-web.onrender.com`	Free
API	`servorabd-api.onrender.com`	Free, sleeps when idle
Database	Neon PostgreSQL, outside Render	Free, no expiry

Verified live: `/healthz/` returns `{"status": "ok", "database": "ok"}`, and the site serves 8 categories, 45 services and 38 locations through the API. Deployed from commit `36d4190` on `main`. The database first lived on Render and moved to Neon when Render deleted it; section 4.6 covers the move.

1.1 The shape of it

```
Browser
   |
   v
servorabd-web  (static site)
   |  /                 index.html, hero prerendered
   |  /providers, /...  app.html, the React app
   |  /api, /admin,     rewritten to the API below,
   |  /static           so the browser sees one origin
   v
servorabd-api  (web service)   Django + gunicorn
   |
   v
Neon           (PostgreSQL, over TLS)
```

The browser only ever talks to the static site. It forwards `/api`, `/admin` and `/static` to Django behind the scenes, which is why no CORS configuration was needed anywhere.

2. What had to change in the code

The repository already had a `render.yaml` and deployment notes, but it would not have survived a first deploy. Five things were fixed, all in commit `36d4190`.

2.1 The build would have crashed

`config/settings/prod.py` required `EMAIL_HOST`, `EMAIL_HOST_USER`, `EMAIL_HOST_PASSWORD`, `DEFAULT_FROM_EMAIL` and the `S3_*` keys. The blueprint marked every one of them `sync: false`, meaning unset until filled in by hand, so Django raised `ImproperlyConfigured` before serving anything. They now have defaults: with no SMTP the mail backend discards messages instead of refusing to start, and with no bucket the storage backend falls back to local disk.

2.2 The health check could never have passed

`healthCheckPath` pointed at `/api/v1/categories/`, but `SECURE_SSL_REDIRECT` is on, so Render's internal HTTP probe received a `301` and the service would have been marked unhealthy forever. A `/healthz/` endpoint was added — it also runs `SELECT 1` against the database — and exempted from the redirect via `SECURE_REDIRECT_EXEMPT`. Confirmed: `/healthz/` answers 200 over plain HTTP while every other path still returns 301.

2.3 Every request would have returned 400

`ALLOWED_HOSTS` was empty on a fresh deploy. It now falls back to Render's own `RENDER_EXTERNAL_HOSTNAME`, which also feeds `CSRF_TRUSTED_ORIGINS`. This is why no host configuration was needed during the deploy.

2.4 Two smaller fixes

- A `/media/*` rewrite was missing. The dev proxy had one; production did not, so uploaded photos would 404 whenever object storage is off — which is the case on free.
- `WEB_CONCURRENCY` was added. The gunicorn config defaults to 3 workers, which is tight on a 512 MB instance.

2.5 Checked and deliberately left alone

The strict Content-Security-Policy was briefly relaxed for Tailwind, then reverted: the built bundle contains no inline script, no inline style and no runtime stylesheet injection, so `style-src 'self'` holds. The full backend test suite passes with all of the above.

3. Two routes that did not work

Worth recording, because both cost time and neither is obvious.

3.1 Blueprint, first attempt: a schema error

Render rejected `render.yaml` with `field command not found in type file.Service` on the two cron services. Render's cron schema uses `startCommand`, not `command`. Both were fixed and pushed.

3.2 Blueprint, second attempt: a demand for a card

With the schema valid, Render asked for payment details. A blueprint is all-or-nothing: Render validates and prices **every** service in the file before creating any of them, and `render.yaml` asks for a paid database and three paid services.

A second blueprint, `render-free.yaml`, was written for the free tier — free database, no cron services, object storage off. It is committed and valid. The card prompt persisted anyway, so the blueprint route was abandoned in favour of creating the services by hand, which sidesteps whole-file pricing entirely.

> **If you return to the blueprint route**
>
> `render-free.yaml` is ready to use. Point the **Blueprint Path** field at it — the field defaults to `render.yaml`, which is the paid one and will be billed. Neither file creates a database any more: paste the Neon connection string into DATABASE_URL when Render asks.

4. The route that worked: services by hand

4.1 Database — a Neon project

The first database was Render's free Postgres, created under **New > Postgres**. Render deletes those 30 days after creation, so it is gone; section 4.6 describes the move. Today the database is a Neon project on the free plan:

Field	Value
Plan	Free — 0.5 GB, no expiry
Region	AWS Singapore, next to the API
Connection string	**Direct**, with connection pooling turned off, ending in `?sslmode=require`

The direct string, not the pooled one: Neon's pooler runs in transaction mode, which breaks the server-side cursors Django uses for large querysets. An idle Neon database pauses after a few minutes and wakes on the next query in well under a second.

4.2 API — New > Web Service

Field	Value
Name	`servorabd-api`
Language	Python 3
Region	Singapore — must match the database
Root Directory	`backend`
Instance Type	Free
Health Check Path	`/healthz/`

Build command:

```
pip install -r requirements.txt \
  && python manage.py collectstatic --no-input \
  && python manage.py migrate
```

Start command — the `../` resolves because Root Directory is `backend`:

```
gunicorn --config ../deploy/gunicorn.conf.py
```

Eight environment variables, which is the whole set needed to boot:

```
DJANGO_SETTINGS_MODULE = config.settings.prod
PYTHON_VERSION         = 3.12.7
WEB_CONCURRENCY        = 1
DEBUG                  = False
USE_OBJECT_STORAGE     = False
TRUSTED_PROXY_COUNT    = 1
DATABASE_URL           = <Neon connection string>
SECRET_KEY             = <Generate>
```

The connection string carries the password. It lives in the Render environment and a password manager, never in the repository.

4.3 Front end — New > Static Site

Field	Value
Name	`servorabd-web`
Root Directory	`frontend`
Build Command	`npm ci && npm run build`
Publish Directory	`dist` — relative to Root Directory, so not `frontend/dist`
Environment Variables	None. Nothing in `src/` reads one; the API is called at the relative path `/api/v1`.

4.4 The five rewrites

Added under **Redirects/Rewrites** on the static site. Without these the landing page loads but every deep link 404s and no data ever arrives.

Source	Destination	Action
`/api/*`	`https://servorabd-api.onrender.com/api/*`	Rewrite
`/admin/*`	`https://servorabd-api.onrender.com/admin/*`	Rewrite
`/static/*`	`https://servorabd-api.onrender.com/static/*`	Rewrite
`/`	`/index.html`	Rewrite
`/*`	`/app.html`	Rewrite

Order matters: `/*` is a catch-all and swallows anything listed below it. The action must be **Rewrite**, not Redirect — a redirect changes the browser's address bar and breaks the single-origin arrangement.

4.5 Seeding

Free web services have no shell, so these were run locally against the Neon connection string, from `backend/` with the virtual environment active:

```
$env:DATABASE_URL = "<Neon connection string>"
$env:DJANGO_SETTINGS_MODULE = "config.settings.prod"
$env:SECRET_KEY = "one-off"
python manage.py seed_catalogue      # 8 categories, 45 services,
                                     # 38 locations
python manage.py createsuperuser
```

createsuperuser does not ask for a name

The user model has `REQUIRED_FIELDS = []` and `full_name` is `blank=True`, so the admin account is created with an empty name and the site header shows nothing. Accounts made through the normal registration form are unaffected. Fix it in `/admin/` > Users, or with `manage.py shell`.

4.6 Moving the database to Neon

Render’s free database reached its 30-day limit and was deleted, and the site lost its data with it. The free plan has no backups, so there was nothing to copy across. The move was a fresh start:

- Created a Neon project as in 4.1 and copied the direct connection string.
- Replaced `DATABASE_URL` on `servorabd-api` with it. Saving the variable redeployed the service, and the build ran every migration against the empty database.
- Ran `seed_catalogue` and `createsuperuser` locally, as in 4.5.
- Signed in to `/admin/` as the new superuser and enrolled its authenticator app.

Checked afterwards: `/healthz/` reports the database `ok`, and the categories and services come back through both the API address and the static site’s rewrite. Nothing in the code changed.

5. How it was verified

5.1 Before deploying, locally

- Production settings load with only the variables Render sets by itself — no SMTP, no bucket, no `.env` file — which is the state of a first deploy. This is the check that caught the crash in 2.1.
- Booted under `config.settings.prod` against a real PostgreSQL database: `/healthz/` returned `ok` and `/api/v1/categories/` returned its rows.
- `check --deploy --fail-level WARNING` clean both bare and fully configured.
- `collectstatic` writes through WhiteNoise to local disk even with object storage on, so the build step needs no bucket credentials.
- `gunicorn.conf.py` resolves from `rootDir: backend` and binds to `$PORT`.
- The frontend build emits both `index.html` and `app.html`.
- The full backend test suite passes.

5.2 After deploying, against the live site

Check	Result
`/healthz/`	`{"status": "ok", "database": "ok"}`
`/`	200, landing page
`/providers`	200, served from `app.html`
`/services`	200
`/admin/login/`	200, styled
Hashed admin CSS	200 — proves WhiteNoise’s manifest storage works
`/api/v1/categories/`	8 categories through the rewrite
`/api/v1/locations/`	38 locations

6. What is still outstanding

6.1 Two things to act on

Delete the expired Render database

`servorabd-db` still appears in the Render dashboard as expired. Nothing uses it. Delete it so no one mistakes it for the live database.

Take your own copies

Neon's free plan keeps only a few hours of history to restore from, so a mistake noticed the next day cannot be undone there. Take a `pg_dump` (6.3) before anything risky and keep it off the server. If the connection string is ever shown on screen or pasted anywhere, reset the role's password in Neon and update `DATABASE_URL`.

6.2 Working as designed, but limited

Limitation	Consequence
No SMTP configured	Login codes are e-mailed, so **nobody can register or log in**. Browsing works; the admin account is unaffected because it is password-based. Set `EMAIL_HOST`, `EMAIL_HOST_USER`, `EMAIL_HOST_PASSWORD` and `DEFAULT_FROM_EMAIL` to switch the mail backend on.
No cron services on free	The seven scheduled jobs never run: bookings do not auto-expire, reviews do not reveal after 14 days, trust scores do not recompute. Run them by hand with `manage.py run_scheduled_jobs`.
No persistent disk	Uploads are written to the container filesystem, which is wiped on every deploy and every wake-from-sleep.
Service sleeps after 15 minutes	The first visit after a quiet period takes roughly 50 seconds. Open the site a minute before showing it to anyone.

6.3 Moving to paid

`render.yaml` already describes the paid arrangement: a service that never sleeps, two cron services, and object storage for uploads. The database stays on Neon; a paid Neon plan adds storage and a longer restore window. Take a copy of the data first:

```
pg_dump --format=custom --no-owner \
        --dbname="<Neon connection string>" \
        --file=servorabd.dump
```

`deploy/RENDER.md` covers the paid route in full, including the object storage bucket and the rate-limiting check that needs a live deployment.

6.4 One check worth doing

Rate limits identify a caller by IP address, read from `X-Forwarded-For`, trusting as many entries as `TRUSTED_PROXY_COUNT` says. Too high a number and a forged header buys a fresh allowance on every request. It is set to `1`, correct for a single Render proxy, but unconfirmed against a live deployment. To check, set `EXPOSE_CLIENT_IDENT=True`, wait for the redeploy, then run locally and set it back to `False` afterwards:

```
python manage.py check_proxy_count \
    https://servorabd-api.onrender.com/api/v1/categories/
```

6.5 Updating the live site

Push to `main`. Both services rebuild, and the API runs its migrations as part of the build. A failed build leaves the previous version serving, so a broken push does not take the site down.